

Order Forecasting: Production Planning Optimization at Oerlikon

Lars Buchholz, Theodor Degen, David Neufang, Leopold Pfeilsticker, and Felix Zukunft

Students at Technical University of Munich

August 1, 2025

Abstract — Whether time-series forecasting can meaningfully improve production planning at Oerlikon is a critical question for cost-efficient and reliable operations. This study investigates the impact of predictive modeling on order forecasting and scheduling in a real-world industrial context, using historical order data from Oerlikon’s coating services. We benchmarked four forecasting approaches: Random Forest, XGBoost, LightGBM, and the Temporal Fusion Transformer (TFT) based on their accuracy and suitability for daily fill rate predictions. Our results show that the TFT model consistently outperforms classical models, and when coupled with a Mixed-Integer Linear Program (MILP), enables significantly better scheduling outcomes, including a 10% increase in machine utilization and over CHF 1.3 million in projected annual savings. These findings highlight the tangible business value of integrating advanced forecasting and optimization tools into manufacturing decision-making.

1 Introduction

This work presents a hands-on analytics project at Oerlikon in which we develop and evaluate a time-series forecasting model for coating service orders, and propose an implementation strategy for integration into production planning. Using two years of historical order data, we applied standard forecasting techniques and time series forecasting mixed with deep learning elements to capture trends and seasonality. Time series forecasting refers to the analysis and modeling of data points ordered in time to predict future values. Capturing the temporal dependencies in such data is pivotal, since past trends, seasonality, and cycles often repeat or evolve. Time-series forecasting methods fit models to historical trends and seasonality so as to anticipate future demands and challenges for manufacturing systems. In practice, this means using past orders to predict the next period’s orders. These techniques are widely applied in industry to guide production scheduling, inventory control, and workforce

planning [7]. By converting raw time-stamped order data into forecasts, manufacturers can proactively adjust capacity and resources. In manufacturing, accurate time-series forecasting has been shown to reduce excess inventory and ensure timely delivery – factors that are directly tied to customer satisfaction [8]. In summary, order forecasting is one application of time-series forecasting where the series is the sequence of past customer orders, and its goal is to drive efficient production planning. This report is structured along the CRISP-DM approach, commonly used in analytics projects. First, the business use cases of a forecasting model are presented to ensure a need for Oerlikon. Section 2 elaborates on Data Understanding which is crucial for model-specific feature engineering steps. Data Preparation follows with the essential feature funnel and data cleaning procedures. To achieve our order forecasts, we used four main models (Random Forest, XGBoost, LightGBM and TFT) which will be explained subsequently in the modeling section. After detailed evaluation and interpretation of the distinct model categories, a Mixed-Integer Linear Program (MILP) is explained to improve scheduling operations at Oerlikon based on the generated forecasts.

2 Business Understanding

This section provides an understanding of Oerlikon’s current operations, the project boundaries and the three identified main business use cases which will underline the relevance of a detailed order forecast of Oerlikon’s coating services business.

2.1 Business Model Canvas

Oerlikon uses coating machines to coat several tool categories (e.g. forming tools) for their customers. After receiving the clients’ tools, the coating is expected to happen within 5 business days. Central to this business is the need to maximize utilization of these capital-intensive assets while ensuring reliable service delivery. Besides other forecasting benefits

(e.g. pricing, sales), maximization of the coaters fill rates is of highest relevance since each coating run incurs a range of setup-related fixed costs. These include electrical energy for heating and operation, labor (either temporary or allocated), wear and tear on the equipment, coating material loss during the start and end of the batch, as well as cleaning and recalibration. Importantly, these costs are largely invariant to the number of items processed per run, making underutilization particularly expensive. Another challenge lies in workforce allocation. While some staff costs may be fixed, such as salaried machine supervisors, other labor elements, particularly around setup and cleaning, can scale with the number of runs. The absence of predictive insight therefore makes it difficult to allocate personnel optimally or minimize the need for overtime and temporary staff. Moreover, coating machines themselves represent high fixed capital expenditures, with lifespans measured in thousands of operational cycles. Every additional run contributes to long-term depreciation and the need for reinvestment. Therefore, consolidating production into fewer, better-filled batches not only improves operational cost efficiency but also extends asset longevity. Finally, a structured forecasting capability can help Oerlikon evaluate whether observed capacity shortages are structural or temporary. This is critical in deciding whether new investments in machinery or facilities are justified, or whether smarter scheduling and planning can resolve the bottlenecks.

2.2 Potential Impact of Order Forecasting

At present, Oerlikon does not employ a formal forecasting model for its order intake. Order scheduling is largely reactive, with production batches typically scheduled immediately upon receipt of an order. This approach prioritizes responsiveness but lacks a systematic mechanism to anticipate short-term demand trends or optimize batch compositions. This often results in suboptimal outcomes, such as partially filled machines being dispatched shortly before new orders arrive that could have been included in the same run. As mentioned, Oerlikon operates under service-level constraints that restrict the flexibility of postponing orders. This underlines the need for predictive tools that allow batching decisions to be made proactively rather than reactively.

Introducing a time-series-based order forecasting system at Oerlikon offers significant potential to improve both cost efficiency and service reliability.

2.2.1 Fill Rate Optimization

The primary lever for value creation lies in improving the average fill rate of machine runs. By forecasting order volumes several days into the future, planners can intelligently group orders into optimally sized batches, thus minimizing the number of low-utilization runs. A modest improvement in average fill rates—e.g., from current levels of 60% up to 70-80% or higher—can yield substantial reductions in per-unit operating costs. Given that fixed setup costs are distributed across a larger output volume, the unit economics improve accordingly. In practical terms, this may translate into 20–30% reductions in the marginal cost per coated part for each run, depending on the process specifics and materials involved. These efficiency gains through fill rate optimization can be reinvested in two ways: either as margin enhancement (by maintaining current pricing) or as competitive positioning (by offering more attractive pricing to customers).

2.2.2 Staff Planning and Sales Improvement

On the staffing side, better predictability allows for more efficient shift planning and potentially lower reliance on short-term or temporary labor. In some settings, this could enable modest reductions in headcount or reallocation of labor to higher value tasks, further amplifying the financial benefits. Finally, a forecast-informed scheduling strategy improves delivery performance. With better visibility into upcoming demand, Oerlikon can plan machine slots more intelligently, reduce lead times, and communicate realistic delivery expectations to customers. This contributes not only to customer satisfaction but also to more stable and manageable production operations.

On the strategic side, improved forecasting supports more confident delivery commitments, strengthens customer relationships, and may even delay or avoid unnecessary capacity expansion. While exact savings depend on local parameters such as machine load profiles, energy tariffs, material types, and labor arrangements, the directional benefits are well supported in manufacturing literature. Studies have consistently shown that predictive planning improves manufacturing performance by enabling “smarter batching, leaner staffing, and more agile responses to demand volatility” [15] [2].

3 Data Understanding

Before proceeding to forecasting and optimization, we conducted an extensive data understanding phase. This analysis aimed to uncover systematic patterns in order volumes and identify key drivers of demand fluctuations. We focused on three core dimensions: intra-week variation, intra-year seasonality, and holiday effects.

3.1 Intra-Week Patterns

A pronounced weekly pattern was evident in the data. Orders were consistently placed only on weekdays, with no orders registered on Saturdays and Sundays. Mondays showed a strong peak in volume, while mid-week activity tapered off slightly. The average daily order volumes per weekday (aggregated over the two years, no orders on weekends) were:

Weekday	Total Orders
Monday	225,500
Tuesday	136,807
Wednesday	120,641
Thursday	127,527
Friday	151,635

Table 1 Total orders per weekday (2023–2024)

This observation justified the inclusion of weekday indicators in the forecasting model. Additionally, the Monday peak indicates possible order backlogs accumulating over the weekend.

3.2 Intra-Year Seasonality

We observed moderate seasonality across months, with dips in December and peaks in March. While some variation exists between years, the general trend holds: Q1 and Q2 months tend to have higher volume than Q4.

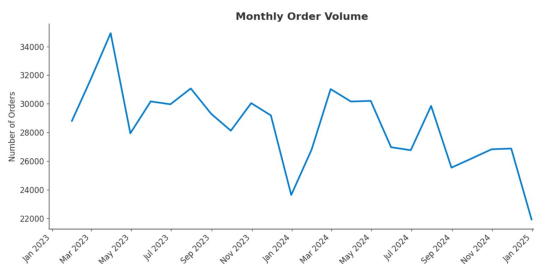


Figure 1 Monthly order volume for 2023 and 2024

Highest and lowest volume months:

- **Highest:** March 2023 (34,934) and March 2024 (30,155)
- **Lowest:** December 2023 (23,631) and December 2024 (21,919)

This confirms a recurring seasonal cycle that is further captured by including monthly and quarterly features in the modeling pipeline.

3.3 Holiday Effects

Initial visual inspection revealed frequent order drops on known public holidays. To systematically assess the impact of public holidays on order volumes, we first identified all dates in our dataset with zero recorded orders across all coating IDs. Cross-referencing these dates with official public holidays in potentially relevant regions and countries, we found a near perfect alignment: Every major holiday in North Rhine-Westfalia (NRW) corresponds to a day of close to zero order activity. This indicates that regional holidays in NRW are a major driver of order shutdowns, e.g. because the team that takes the orders is seated there. Other dates with zero or low orders were not explained by non-NRW holidays.

For a more nuanced analysis, we employed an ordinary least squares linear regression to quantify the potential effect of non-NRW holidays on daily order volumes. We regressed the daily total order volume against a binary indicator for “holiday in any other region” and included weekday and month fixed effects to control for seasonality. The coefficient for the non-NRW holiday variable was both statistically insignificant and close to zero, indicating no meaningful reduction in order volume attributable to these holidays.

4 Data Preparation

Within this section, the performed steps of data preparation are described. Firstly, data cleaning was performed, followed by feature engineering and finally the clustering of Coating IDs.

4.1 Data Cleaning

The following models were trained on a preprocessed dataset. The raw Oerlikon data consists of 801919 order records, spanning from January 2023 until beginning of April 2025. Since Oerlikon offers 161 different coating materials, this results in possible 161 order types to predict. However, due to outliers, negative fill rates, missing values and negative prices, several cleaning

steps were conducted so that model requirements are fulfilled.

1. We removed rare coating IDs by keeping only the most frequent 95% based on their relative occurrence. This helped reduce noise in the model. Including rare coating IDs typically leads to artificially low but misleading mean absolute errors (MAE), since the model learns to predict zeros—which may be technically accurate but add little value and obscure actual forecasting errors.
2. Entries with missing values in essential columns such as `Industry`, `Product_Family`, `Customer_ID`, and `fill_rate` were dropped to ensure data completeness.
3. We excluded negative values in the columns `CHF` and `fill_rate`, as both revenue and machine utilization are expected to be non-negative.
4. The `order_date` column was parsed and converted to a proper date format to enable correct temporal grouping and analysis.
5. We aggregated the target variable `fill_rate` by summing daily values per `coating_id`, creating a meaningful target for time-series forecasting.
6. To enrich the dataset with external knowledge, we merged calendar-based data such as public holidays. This allowed the model to potentially capture effects of work-free days on order behavior.
7. Because the raw dataset only included records for days with orders, we reindexed the data to include all possible day–product combinations. For days without orders, missing numerical values were filled with zero, and missing categorical values were filled with “None.” This correction was critical to avoid biasing the model toward busy days and to better reflect actual production conditions.

4.2 Feature Engineering

Feature engineering plays a critical role in machine learning application because the performance of machine learning models is heavily influenced by how data is represented. It allows to expose central factors of variation, enhance generalization, and improve task performance. A precise selection of features can lead to better abstraction and invariance to irrelevant variations which facilitates learning and recognition of patterns. [3]

In our case, although the dataset provided by Oerlikon was extensive, its raw form did not reflect trends or temporal patterns relevant for forecasting. To tackle

this limitation, we used insights gained during the data understanding phase to engineer six main groups of features:

1. **Lag Features:** Previous observations of key variables were transformed into input features to capture autocorrelation and temporal dependencies.
2. **Temporal Features:** Time-based indicators such as weekday, month, quarter, and business cycle phases were included to account for seasonality and periodic effects.
3. **Rolling Statistics:** Moving averages, sums, and standard deviations were computed over multiple window sizes for variables such as fill rate, revenue, and customer ID to capture local trends and variability.
4. **Customer Behavior Features:** Metrics were developed to identify and track the most influential customers based on sales volume and frequency.
5. **Exponentially Weighted Averages (EWA):** EWAs were calculated to give higher importance to recent observations, thereby improving sensitivity to short-term changes in customer behavior.
6. **Calendar Effects:** Features were constructed to compare performance metrics on the same relative weekday of previous months, quarters, or years, aiming to capture recurring calendar-driven patterns.

To develop the above features, the Python library `tsfresh` was used. The `tsfresh` (Time Series Feature Extraction on basis of Scalable Hypothesis tests) package is designed to automate the process of feature engineering for time series data. It extracts a large number of time series characteristics and evaluates their relevance using scalable hypothesis testing. The library operates in three main stages: feature extraction, p-value computation for each feature with respect to the prediction target, and feature selection using a multiple hypothesis testing framework. By automatically configuring statistical tests based on the types of features and target variables, `tsfresh` facilitates the identification of the most informative features and helps reduce the risk of overfitting [6].

`tsfresh` assisted in setting up lag, rolling features, resulting in 180 features. The remaining feature groups like calendar and customer-related features were developed manually. Especially, representing holidays correctly and encoding cyclical features required precise modeling.

The resulting feature base consisted of a total of 736 features. To reduce redundancy and ensure that dominant features did not overshadow others, this base was

evaluated using three different machine learning models: LightGBM, XGBoost, and Random Forest. These models include built-in mechanisms for evaluating feature importance. Based on the resulting importance scores, three thresholds were defined: in the first case, all features with an importance score less than or equal to 0 were removed; in the second, a threshold of 100 was applied; and finally, a more selective threshold of 220 was used.

Name	Threshold	Resulting Feature Set Size
Feature Set 1	0	749
Feature Set 2	100	250
Feature Set 3	220	50

Table 2 Feature set sizes after applying feature importance thresholds

All three feature sets were then used during the modeling phase to evaluate their impact on training time and performance metrics.

4.3 Clustering of Coating IDs

To improve predictive performance and account for heterogeneity in product demand patterns, we performed a clustering analysis to group coating IDs according to their characteristics. The rationale for clustering was to enable the training of specialized forecasting models within more homogeneous product segments, rather than fitting a single model across all coatings with widely varying behavior. Alternatively, the cluster can also be used as an additional feature. We explored two complementary clustering pipelines. The first approach involved aggregating the features for each coating ID, standardizing and reducing their dimensionality with principal component analysis (PCA), and then applying k-means clustering to group coatings with similar statistical behavior. In a second, more model-driven variant, we computed "SHAP fingerprints": After training a global LightGBM forecasting model on all coating IDs using our largest feature set, we calculated per-ID mean absolute SHAP values, quantifying the importance of each feature for each product. These SHAP-based importance profiles were then normalized, scaled, PCA-reduced, and clustered to reveal groups of coating IDs with similar forecast-driving factors. This SHAP-fingerprint clustering enabled us to go beyond raw time-series statistics and segment products according to their response to predictive signals. For a more detailed explanation of our approach as well as the result of the clustering analysis, please refer to the

code and its documentation. Ultimately, we adopted the latter approach for our final modeling pipeline, as it provided more meaningful groupings for specialized forecasting, reflecting the true drivers of demand for each product cluster.

5 Modeling

In this section, the modeling approach is described. Firstly, we decide on the required forecasting horizon. Afterwards, different models and prediction levels were defined and compared to each other.

For our project we decided to evaluate the following models: Random Forest, XGBoost, and LightGBM, which are all tree-based models, and Temporal Fusion Transformer, a deep learning time-series model.

5.1 Forecasting Horizon

A key consideration in our modeling strategy was the selection of an appropriate forecasting horizon. While multi-horizon forecasts (e.g., predicting 5 days ahead) offer potential strategic value for longer-term planning, they are subject to an inherent trade-off between forecast length and precision [16].

Given that for further evaluation, such as a daily production scheduling optimization, we need the most accurate input possible, we evaluated models with different time horizons. Our results, which will be detailed in the 6th chapter, demonstrated that the 1-day-ahead forecast achieved the lowest error rates. Consequently, to maximize the operational value and reliability of the downstream optimization task, a 1-day forecast horizon was selected for all final models.

5.2 Random Forest (RF)

Random Forest is an ensemble learning method that builds a collection of decision trees and aggregates their predictions to enhance robustness and generalization. It was originally introduced by Breiman (2001) as a way to improve predictive accuracy while reducing overfitting through the use of bagging (bootstrap aggregation) and randomized feature selection at each tree split [4].

In regression settings, each tree outputs a continuous prediction, and the forest combines them typically via mean averaging. Key hyperparameters include the number of trees, tree depth, feature subset size, and minimum sample splits per node which were tuned in our implementation to match the characteristics of the dataset and control model complexity.

In the context of our application to predicting the next-day fill rate per Coating ID from time-series derived features, the Random Forest Model was selected as a baseline and due to several theoretical and empirical reasons. First, Random Forests are well-suited for structured tabular data with mixed feature types and nonlinear dependencies [12]. Our input space consists of over 1,000 manually engineered features capturing lagged values, rolling statistics, calendar effects, and exponentially weighted averages (EWAs). These features do not follow a simple linear structure but exhibit hierarchical, conditional, and temporal interactions. Here Random Forests can learn due to their non-parametric decision-tree-based structure [20]. Unlike linear models, Random Forest does not require feature standardization or pruning of correlated variables, as individual trees handle these redundancies through feature bagging [4]. This is particularly relevant in our case, where rolling-window and lag features may be partially collinear. Following, a significant advantage of Random Forests is the ability to compute feature importances, which measure how much each variable contributes to prediction reduction in node impurity (e.g., via Gini importance or permutation-based metrics). In our pipeline, this interpretability was critical for ranking the features and creating three reduced feature sets (749, 250, 50 features) via importance-based thresholds. This helped simplify the modeling process and improve downstream generalization, while also allowing domain experts to understand which temporal or behavioral variables were influential.

Given that each Coating ID may have distinct ordering patterns and temporal dynamics, we implemented Random Forest models at both the global level (across all IDs) and the per-ID level. Random Forest’s relatively low training time and independence of time order made it feasible to train and evaluate a large number of localized models. This architecture helped us to test whether some Coating IDs could benefit from dedicated model attention due to specific customer behaviors or usage cycles.

5.3 Extreme Gradient Boosting (XGBoost)

XGBoost (eXtreme Gradient Boosting) is a scalable and regularized ensemble learning technique based on gradient boosted decision trees. Originally introduced by Chen and Guestrin (2016), it builds models in a stage-wise fashion, where each successive tree attempts to minimize the residual errors of its predecessors using a differentiable loss function [5]. Unlike bagging

techniques, boosting iteratively improves weak learners by sequentially optimizing the model toward the most difficult-to-predict samples, thus often achieving high predictive accuracy [17].

Given the engineered time-series structure of our dataset and the complexity of forecasting fill rate behavior across Coating IDs, XGBoost was selected for several compelling reasons. XGBoost is particularly powerful in capturing nonlinear relationships and hierarchical feature interactions, which are highly likely in our dataset due to the combination of lag features, rolling statistics, customer behavior, and calendar encodings. While Random Forest treats each tree as independent, XGBoost builds trees in sequence, allowing it to iteratively refine its predictions [17]. This could be a valuable property when modeling subtle interactions in temporal data, such as how fill rates vary by weekday or across holiday seasons as we found in our data understanding. Therefore, this could make XGBoost especially promising for handling heterogeneous patterns across Coating IDs without the need for deep learning models. One of XGBoost’s key innovations over traditional boosting methods is its incorporation of L1 and L2 regularization into the loss function, helping prevent overfitting in high-dimensional settings [23]. Given our dataset initially included over 1,000 features, many of which may be weakly predictive or correlated, this helped for ensuring stability and generalization. This is especially important in smaller training segments when modeling per Coating ID. In addition, hyperparameters like `subsample` and `colsample_bytree`, used in our implementation, introduce stochasticity into the model and further control variance. Similar to Random Forests, XGBoost provides feature importance metrics based on gain, cover, or frequency. This enabled us to rank features by predictive contribution. These scores were used to derive the three feature sets for benchmarking (base, 250, 50), which improved interpretability and allowed us to reduce dimensionality when needed.

5.4 Light Gradient Boosting Machine (LightGBM)

Finally, LightGBM is a gradient boosting framework developed by Microsoft. It implements a novel approach to gradient boosting decision trees (GBDT) using histogram-based feature binning and a leaf-wise tree growth strategy [22]. This makes it a fast and memory-efficient method. These structural innovations allow LightGBM to grow deeper trees with fewer splits and to focus model complexity more selectively

in regions of high loss [10]. In our setup, where time-encoded features and behavioral metrics vary substantially across Coating IDs, this allows LightGBM to concentrate learning on high-variance, high-signal regions of the data which is something standard depth-wise boosting may underexploit.

5.5 Temporal Fusion Transformer (TFT)

While the previous tree-based models provided a robust performance baseline, their architecture is fundamentally static. They treat each forecast as an independent prediction [13], which is why the use of lagged features was essential. This approach, however, has limitations for complex time-series tasks similar to our given case: they do not natively distinguish between static metadata, time-varying past inputs, and time-varying known future inputs, all of which are present in our dataset.

To overcome these limitations, we decided to use a model which is temporally aware, such as Temporal Fusion Transformers (TFT). The TFT is a state-of-the-art deep learning architecture introducing the attention mechanism, which enabled the breakthrough for large language models, for multi-horizon time series data [19]. As a main characteristic, it simultaneously learns from different types of temporal features and identifies long-range dependencies in the data, which is crucial for capturing complex seasonal patterns [19] (see section 4). In contrast to simpler statistical time-series models like ARIMA, TFT can also handle additional features, which stem, for example, from our dataset [16]. This means that we have to create only one model and then have to use the coating id as a static input value.

5.5.1 Theoretical Foundations and Architecture

The architecture of the Temporal Fusion Transformer (see Figure 10 in Appendix), was created to address common challenges of real-world time series forecasting. At a high level, the model processes data in a sequential flow: (1) It utilizes three types of input: static metadata, past time-varying inputs, and known future time-varying inputs. (2) A series of variable selection networks learns the relevance of each input feature. (3) A sequence-to-sequence architecture that uses Long Short-Term Memory (LSTM) to encode past information and decode future information, captures temporal dynamics. (4) A temporal self-attention mechanism identifies and weights the most relevant historical time steps for the forecast. (5) The processed

information is passed through a dense layer to produce a quantile forecast.

The following sections will dissect each of these core components in further detail:

- **Gated Residual Network (GRN) as adaptive information filtering:** By using a gating mechanism, the model can learn to dynamically suppress or amplify certain features based on the context provided by other inputs [19] (section 4.1). This enables the network to filter out irrelevant or noisy information.
- **Variable Selection Networks for learning feature importance:** For each input type, Variable Selection Networks, which are based on GRNs, enable the model to focus its capacity on the most predictive features and to ignore uninformative ones, which removes the manual step of feature selection [19] (section 4.2). Additionally, it creates the opportunity to evaluate those interpretable weights.
- **Sequence-to-Sequence LSTMs for capturing temporal dynamics:** There are two LSTMs: the encoder LSTM processes the sequence of past inputs to create a compressed representation of the historical context, while the decoder LSTM uses this context to combine it with the sequence of known future inputs. This helps the model to learn short-term temporal patterns such as trends and seasonality.
- **Interpretable Multi-Head Attention for identifying relevant time steps:** One disadvantage of LSTMs is that they do not excel in capturing long-range dependencies, for example due to the vanishing gradient problem [14]. For this reason, TFT includes a multi-head attention mechanism, which became a known component in the Transformer architecture [24]. It allows the model to look back across the entire input sequence and to calculate an attention weight for each past time step. In this way it can identify specific, influential events that are relevant for the current forecast.
- **Quantile Forecasts: Quantifying Uncertainty:** The TFT was designed to produce quantile forecasts, which consists of multiple points of the future distribution (e.g., the 10th, 50th, and 90th percentile). If enabled, it allows the construction of prediction intervals (e.g. 0.95 confidence interval), providing a measure of the forecast's uncertainty which can be useful for informed decision-making in production and staff planning.

5.5.2 Implementation and Training

Regarding the data preparation for this model, we had to create a pipeline solely for this model. The reason was that TFT, being a time-series model, requires a specific data structure. This includes a temporary continuous time index (filling weekends with 0 values, which will be excluded for the calculation of performance metrics), generating time-varying features (like lags and rolling averages) and distinguishing between static, past-known, and future-known variables. We only used a subset of the features we used for the previous three models as the training of TFT was computationally heavy.

We then aggregated the daily fill rate per coating id in a similar fashion as for the previous models. This target variable was additionally normalized. We decided against using the standard z-score normalization as our daily fill rate was strictly non-negative and its distribution was highly skewed. For this reason, we implemented a Log1pNormalizer ($\log(1+x)$) which stabilizes the variance of the series and makes its distribution more Gaussian, which is beneficial for the model's learning process [11]. The corresponding inverse transformation ($\exp(x)-1$) ensures that the model predictions are constrained to be non-negative, which reflects our real-world business constraint.

Lastly, we removed data for coating ids which had a low amount of orders in the historical data, as this led the model to always predict zero fill rates, artificially improving the overall model performance. This led to our Time Series Data Set.

Using machine learning models also adds the need of hyperparameter optimization, a process where we try to identify the optimal set of hyperparameters. Hyperparameters are values which affect the way a model learns [11]. They can be set to fixed values, or we can try out different combinations. As many of those features are continuous values, it is impossible to try out all possible combinations. For this reason we used Optuna, a Bayesian optimization framework, which systematically searches for the best hyperparameters [1]. We focused on the following hyperparameters:

- **Learning Rate:** This parameter determines the step size at each iteration while updating the model parameters [11].
- **Hidden Size:** This parameter determines the capacity of the network [11].
- **Attention Head Size:** This parameter determines the amount of separate temporal patterns the model can simultaneously learn [24]. Lower numbers lead to the model being able to abstract

more general patterns, while higher numbers lead the model to specialize on specific patterns.

- **Dropout:** This is a regularization technique which randomly disables a specified amount of neurons during each training step, leading the model to not rely on crucial neurons [21].
- **Batch Size:** This is the number of time series sequences that a model processes at once before it updates its weights [11].
- **Maximal Epochs:** This is the number of times the model is allowed to see the entire training dataset [11]. Often, this number isn't reached, as the early stopping we implemented stops the training once it determines that the model isn't improving anymore.

To speed up the process of finding optimal hyperparameters, we also used a hyperband pruner which makes the search more efficient by early stopping the training of unpromising hyperparameter combinations early [18].

The final hyperparameter combination can be found in the appendix ().

5.5.3 Deciding on overall prediction settings

For the purpose of this task, we trained a couple of TFT models.

Possible decisions we had to take based on our goals were regarding the loss function, the time horizon of our prediction, and the length of the historical context the model is allowed to use for a prediction.

During training, we discovered that models with a mean absolute error loss, one-day-ahead forecast, and 60 days historical window performed the best. We will therefore use these settings for the architecture comparison and for the Mixed-Integer Linear Program.

For quantile loss, the model is trained to optimize all output values as explained before. With MAE loss, the model is only trained to optimize the single best guess of the future fill rate. Using a MAE loss therefore leads to better results for our target value while losing uncertainty information.

The results of a selection of models we trained can be found in the appendix. We still believe that other settings, like the uncertainty estimation and longer prediction windows, are useful for different business decisions.

6 Evaluation

To evaluate model performance, we selected mean absolute error (MAE) and the coefficient of determination (R^2) as our primary metrics. MAE reflects the average deviation between predicted and actual values and is useful for understanding raw prediction error. However, MAE alone is not suitable for comparing Coating IDs, since some IDs exhibit long idle periods with fill rates near zero. This results naturally in artificially low MAE even for weak models. To counterbalance this, we introduced R^2 , which compares the model's predictions against a naive mean predictor. This allows us to assess the true explanatory power of each model relative to a simple baseline.

6.1 Evaluation of the tree-based models

In this section we evaluate the tree based models based on the respective feature sets. Additionally, we differentiate between the individual models for each Coating ID, the clustered model with four clusters, and the single model where Coating ID is simply an additional input value.

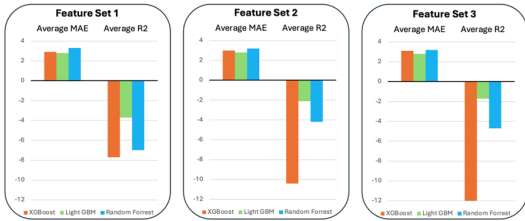


Figure 2 Individual Model per Coating ID

Across all three feature sets, individual models trained per Coating ID yielded consistently poor R^2 scores, often negative (See Figure 3). This indicates that they performed worse than a naive baseline. This suggests that modeling each Coating ID separately leads to data fragmentation, where insufficient training data per ID prevents the models from generalizing effectively. Notably, XGBoost degraded the most with increasing feature sparsity (Feature Set 3), while LightGBM and Random Forest remained more stable but still underperformed.

By looking at the metrics of the model which utilizes clusters, we can see a slight improvement, which means that unifying the fill rate for similar Coating IDs improved the prediction. Still, we haven't achieved a level yet which is suitable for further predictions. (WEERTE FEHLEN)

In contrast, when a single global model as visualized in Figure 4 was trained across all Coating IDs, perfor-

mance improved markedly. All three classical models (XGBoost, LightGBM, and Random Forest) achieved positive R^2 scores, with MAE dropping by roughly 1 unit compared to the per-ID setup. Among the feature sets, Feature Set 2 offered the best trade-off.

This comparison reveals two key insights: (1) local per-ID models suffer heavily from limited data, and (2) shared models that leverage common temporal patterns across coatings are significantly more effective, even for non-sequential regressors. It also highlights the importance of data aggregation for robust predictive modeling.

6.2 TFT Performance and Interpretation

This section presents an evaluation of the Temporal Fusion Transformer model. First, we assess the model's predictive accuracy by comparing it against the tree-based benchmarks. Second, we utilize the generated interpretability features to qualitatively analyze the model's internal logic, identifying significant drivers of the daily fill rate.

6.2.1 Comparative Predictive Analysis

Across all tested feature sets, our TFT model consistently outperformed all other models with respect to MAE and R^2 .

For the optimal feature set, the TFT model achieved a MAE of 2.23. This represents a 7.1% reduction compared to the next-best model, LightGBM (MAE of 2.4). Furthermore, the TFT's R^2 value of 0.63 indicates that it explains 63% of the variance in the target variable, which is a substantial improvement over the tree-based models, whose R^2 values did not exceed 0.48. Those are the overall metrics, meaning that for some coating ids the model performed better, while for others it performed worse.

6.2.2 Model Interpretation

One advantage of the TFT architecture is its built-in interpretability, which allows us to analyze how the model came to its conclusion. In this model, features were categorized into three categories: static variables, future-known variables used on the decoder and historical variables used in the encoder.

Looking at the analysis of static variables (Figure) we can see that the most influential features is the coating id itself. This confirms that the model has learned that each coating id has its own unique, baseline demand pattern. We can also see that using the scaled daily fill

rate, which represents the typical magnitude of sales for each coating id, is the second most important variable. This makes sense as knowing the general sales volume of a Coating ID is a powerful predictor for the scale of its future demand.

By looking at the decoder variables (see Figure), which represent calendar and event information known in advance, we see that the day of the week was the most influential factor. This aligns with our discovery that there is a repetitive pattern across all weeks, as for example the orders on Mondays were the highest. We also see that features which identify business days within a month play a crucial role. This likely corresponds to corporate purchasing cycles.

Lastly, the encoder variables (see Figure), which represent the historical context, indicate that there is a monthly pattern in the data. Additionally, we also see that the model heavily relies on recent performance, with the historical daily fill rate and short-term lags being highly influential.

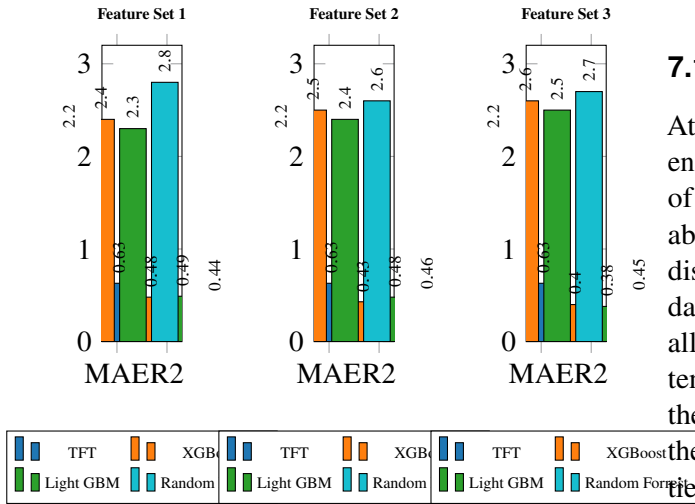


Figure 3 Comparative Performance of all models on the validation set

7 Deployment

While demand forecasting can support decision making in many business domains, we focus on its benefits for production scheduling. To translate demand forecasting into actionable production plans, we deploy a two-layer decision pipeline at Oerlikon. First, a time-series forecasting model (described in Section 3) generates daily demand predictions for each coating ID. Second, a scheduling system consumes these forecasts as well as actual demand and backlog data to generate cost-optimal machine-specific coating schedules.

In this section, we introduce a Mixed-Integer Linear Program (MILP) specifically developed for Oerlikon’s scheduling requirements (Section 4.1) and assess the performance of our scheduling approach against the current scheduling approach using real operational data (Section 4.2).

7.1 Production Scheduling via MILP

Our scheduling system centers on a Mixed-Integer Linear Program, designed to convert demand forecasts into actionable daily production plans for Oerlikon’s coating operations. This is based on the MILP formulation of Floudas and Lin. MILPs are widely used in industrial scheduling due to their ability to model discrete decisions, incorporate operational constraints, and provide globally optimal solutions under reasonable computational times [9]. Based on forecasted and actual demand, the MILP determines the optimal production quantities for each product over a rolling five-day planning horizon.

7.1.1 Core Assumptions and Design

At its foundation, our MILP enforces strict due dates, ensuring that every order is completed within five days of arrival, upholding Oerlikon’s commitment to reliable customer deliveries. Production is organized in discrete batches, each machine operating under fixed daily capacity constraints. Importantly, machines are allowed to switch freely between coating IDs, and potential setup or changeover times are absorbed within the batch definition, eliminating the need to model them explicitly. To further align with operational realities, we assume that batches may be scheduled during regular working hours or overtime, with overtime shifts incurring higher personnel costs. The model also enables early deliveries: orders can be completed before their due date, increasing overall capacity utilization when possible. Storage costs are not considered. A First-In, First-Out (FIFO) discipline ensures that older backlog is prioritized, thus preventing the aging of orders and maintaining fairness across products.

7.1.2 Objective Function

The MILP’s objective function minimizes the total cost associated with scheduling while incentivizing early deliveries. It incorporates three key components:

- **Regular and overtime production costs**, in particular staffing expenses including opportunity costs;

- **Bonuses for early delivery** (modeled as negative costs), incentivizing proactive scheduling and providing flexibility and robustness to the system;
- **Penalties for unprocessed backlog**, ensuring all customer commitments are met by discouraging unscheduled orders at the end of the horizon.

This objective, subject to all operational and capacity constraints, allows the model to recommend efficient and feasible daily production schedules that are both cost-effective and reliable. For the complete mathematical formulation, all variables, and detailed constraints, see Appendix A.

7.2 Model Evaluation and Results

7.2.1 Experimental Setup

To assess the effectiveness of our MILP-based scheduling approach, we benchmarked its performance against Oerlikon’s current scheduling method, which assigns each order to the next available slot without batching or overtime planning. The evaluation used operational data from March 2025 for the 30 biggest coating IDs. To ensure a fair comparison and to eliminate initial transients, a 3-day warm-up and a 3-day cooldown period were applied; only results from the central 15-day window were considered (see code for details).

Due to a lack of fixed machine availability information, we compared performance and variable costs across different hypothetical numbers of machines. Each day, the MILP was re-solved using actual demand for the day, the forecast for the following day, and historical averages for subsequent days (up to five days ahead). Multi-day-ahead forecasts could also be incorporated but are not included in this main analysis. The MILP was solved independently for each coating ID, and only the schedule for the current day was implemented before rolling forward.

The solver time per run was limited to 10 seconds to ensure practical feasibility; if the optimal solution was not found within this limit, the best available solution was accepted. In all cases, the optimality gap remained below 5%. The entire implementation was realized in Python with **Gurobi 11.0.0** as the optimization engine.

7.2.2 Sensitivity Analysis and Comparative Results

To assess the impact of forecasting quality on scheduling performance, we first evaluated the MILP using outputs from our different forecasting models as well as a simple historic average until March 2025 and compare

the performance to the current scheduling approach, assuming 50 available machines. As shown in Figure ??, scheduling decisions of our MILP based approach significantly outperformed the current scheduling practice at earlicon. It can also be observed that using any of our forecasts as input for the MILP outperforms only using historical data. Additionally, the one-day-ahead prediction from the TFT model yields the lowest total personnel costs at **CHF 288,000** for the 15-day period. By contrast, using the multi-day-ahead TFT forecasts resulted in **CHF 289,760**. Models that achieve higher accuracy for high-volume coating IDs tend to produce the most cost-effective schedules, as batching for these products is critical to fill rates. For low-volume IDs, combining all available orders is typically optimal, since full batch utilization is rarely possible regardless of prediction accuracy.

In a second analysis, we compared the MILP approach (using TFT forecasts) to Oerlikon’s current scheduling practice across varying machine numbers. As summarized in Figure ??, the MILP approach consistently outperformed the baseline: Compared to the current practice at Oerlikon, average fill rates increased by **10 percentage points** (from 60% to 70%), while total personnel costs were reduced by **CHF 77,120** over the 15-days evaluation period, with projected annual savings exceeding **CHF 1.3 million**.¹ The number of overtime batches was also substantially reduced (see Figure ??).

7.2.3 Business Impact

Beyond the quantifiable cost and utilization gains, the MILP-based approach delivers important operational advantages for Oerlikon, including greater planning flexibility, enhanced schedule reliability, and improved on-time delivery. These improvements strengthen customer communication, reduce churn risk, and support higher sales through better fulfillment rates.

While this paper focuses on production scheduling, it is important to note that advanced demand forecasting can create value across multiple business domains enabling more targeted marketing, adaptive pricing, churn prediction, and supporting strategic decisions such as facility expansion or location planning.

For further details, including full data, parameter settings, and experimental workflow, see Appendix B.

¹Derived from personnel cost: 40 CHF/hour, 7 hours per batch (6 h production + 1h setup), with 1 worker for 2 machines. Calculation: $(40 \text{ CHF/h} \times 6 \text{ h} \times 40 \text{ CHF/2}) = 160 \text{ CHF}$ - Overtime cost is assumed to be twice as high.

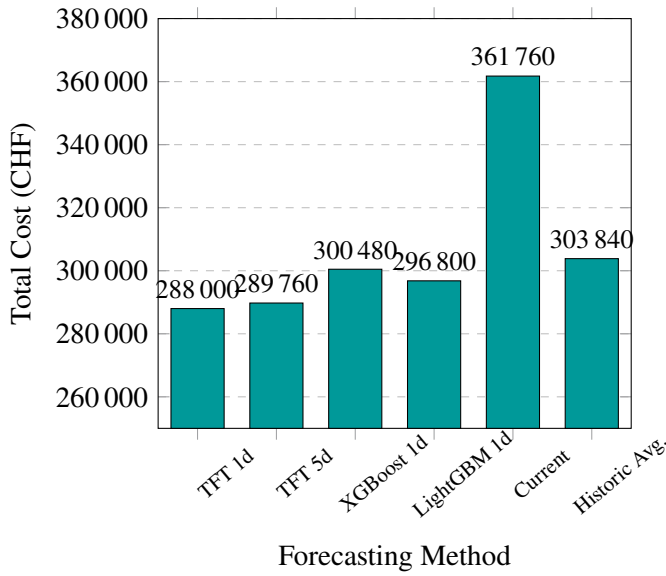


Figure 4 Total personnel costs for different forecasting models (50 machines, 15 days).

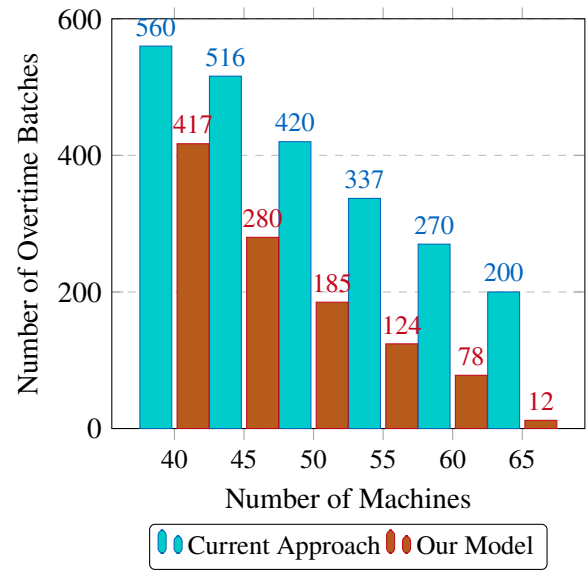


Figure 6 Number of overtime batches for the current scheduling approach and our MILP model by machine configuration.

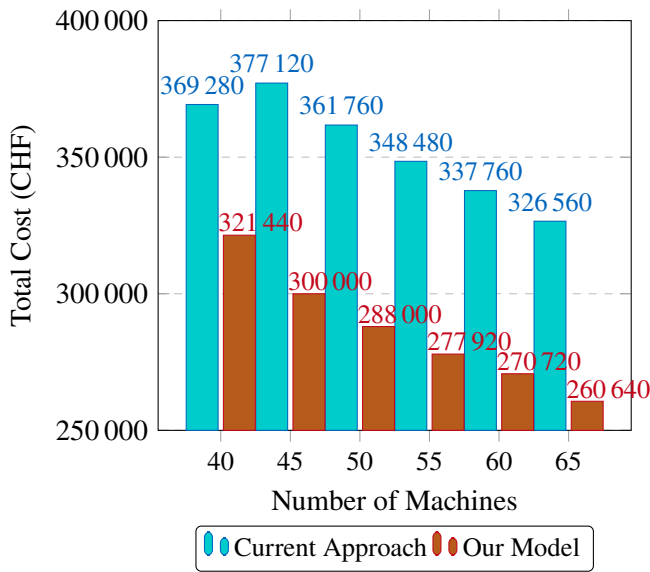


Figure 5 Total costs for the current scheduling approach and our MILP model by machine configuration.

architecture for the short-term forecasting of coating orders.

These forecasts serve as critical inputs to the MILP. By systematically batching orders and minimizing overtime usage, the MILP achieves measurable cost savings while improving average machine fill rates. Compared to Oerlikon's current scheduling strategy, our integrated TFT + MILP approach reduced total personnel costs by CHF 77,120 over 15 days, projecting annual savings exceeding CHF 1.3 million.

Looking ahead, **Oerlikon can incorporate its daily TFT forecasts into the MILP framework**, enabling dynamic and cost-efficient production planning. The system provides flexibility to scale with fluctuating demand and can be extended to multi-day forecasts or uncertainty-aware scheduling. Beyond immediate per-batch cost reductions, this approach enables better customer communication, facilitated staff planning, and more informed strategic decisions.

8 Conclusion and Outlook

9 Conclusion and Outlook

This project demonstrates that machine learning-based order forecasting, combined with optimization-based scheduling, can yield significant operational improvements for Oerlikon. Among all evaluated models, the **Temporal Fusion Transformer (TFT)** showed the highest predictive accuracy, outperforming tree-based models in both MAE and R^2 metrics. With a MAE of 2.23 and $R^2 = 0.63$, TFT emerged as the most suitable

Appendix X: ...

X.Y TFT comparison

	Loss	Days ahead prediction	Max Trials	MAE	R ²
1	MAE	1	45	2.25	0.68
2	MAE	1	15	2.23	0.63
3	Quantile	1	15	2.31	0.60
4	Quantile	5	15	2.44	0.38

Table 3 Comparison of the performance of a selection of TFT models we trained. We decided to use model 2 as it had the best performance with respect to MAE. Model 1 was trained under the same conditions, with the exception of having more Optuna trials. Model 5 shows the average performance metrics over all predicted time periods.

	Attention Head Size	Batch Size	Dropout	Hidden Size	Learning Rate	Max Epochs
1	3	128	0.1320	128	0.0017	64
2	2	64	0.1015	64	0.0013	32
3	2	64	0.1750	32	0.0014	32
4	4	128	0.3097	32	0.0037	32

Table 4 List of hyperparameter for a selected number of TFT models. The model numbering is the same as for Table 3. Hyperparameter were rounded to four decimals, if necessary. The exact values can be found in the GitHub reports in the respective sample reports.

Hyperparameter	Description
Learning Rate	"This parameter determines the step size at each iteration while updating the model parameters [11]."
Hidden Size	"This parameter determines the capacity of the network [11]."
Attention Head Size	"This parameter determines the amount of separate temporal patterns the model can simultaneously process [11]."
Dropout	"This is a regularization technique which randomly disables a specified amount of neurons during training [11]."
Batch Size	"This is the number of time series sequences that a model processes at once before it updates its weights [11]."
Maximal Epochs	"This is the number of times the model is allowed to see the entire training dataset [11]. Often, this is determined by a patience parameter [11]."

Table 5 Description of the hyperparameter which were selected in the training process of the TFT model. If Optuna was enabled for determining the hyperparameter, it sampled values out of the given areas.

Appendix A: MILP Formulation

A.1 Objective Function

The MILP aims to maximize net profit by balancing scheduling costs, overtime, and early delivery benefits:

$$\min \sum_{i \in I} \sum_{t \in T} \left(c_{\text{reg}} \cdot n_{i,t}^{\text{reg}} + c_{\text{ot}} \cdot n_{i,t}^{\text{ot}} \right) - \sum_{i \in I} \sum_{t \in T} \sum_{a \in A} \gamma \cdot p_{i,t,a} \cdot (4 - a) + \sum_{i \in I} \lambda \cdot s_{i,4} \quad (1)$$

Decision Variables

- $n_{i,t}^{\text{reg}}$: Number of regular batches of coating i on day t
- $n_{i,t}^{\text{ot}}$: Number of overtime batches of coating i on day t

- $p_{i,t,a}$: Volume of backlog of age a for coating i coated on day t
- $s_{i,t}$: Unprocessed backlog of coating i at day t

Key Parameters

- I : Set of coating product IDs
- $T = \{0, \dots, 4\}$: Planning horizon (days)
- $A = \{0, \dots, 4\}$: Backlog age buckets (0 = new order, 4 = last day before due)
- $c_{\text{reg}} = 160$: Cost of a regular batch in CHF
- $c_{\text{ot}} = 320$: Cost of an overtime batch in CHF
- $\gamma = 50$: Early delivery bonus per unit and day
- $\lambda = 100$: Penalty per unprocessed unit
- $Q = 1$: Volume capacity per batch
- $M = 65$: Number of coating machines
- $B_{\text{reg}} = 2$: Regular batches per machine per day
- $B_{\text{ot}} = 1$: Overtime batches per machine per day
- Regular staff cost per hour: 40 CHF
- Overtime staff cost per hour: 80 CHF (2× regular)
- Batch time (regular and overtime): 6 hours + 1 hour setup

A.2 Constraints

The discussed MILP model is subject to the following constraints:

1. Backlog Update Constraints (Order Flow):

$$s_{i,t+1,a+1} = s_{i,t,a} - p_{i,t,a}, \quad \forall i \in I, t \in T, a \in A \setminus \{4\} \quad (2)$$

$$s_{i,0,0} = d_{i,0}, \quad \forall i \in I \quad (3)$$

2. Process Only Existing Orders:

$$p_{i,t,a} \leq s_{i,t,a}, \quad \forall i \in I, t \in T, a \in A \quad (4)$$

3. Due-Date Clearance (No processing after day 4):

$$p_{i,t,a} = 0, \quad \forall i \in I, t + a > 4 \quad (5)$$

4. Batch-Volume Link (Each batch has fixed volume):

$$\sum_{a \in A} p_{i,t,a} \leq Q \cdot (n_{i,t}^{\text{reg}} + n_{i,t}^{\text{ot}}), \quad \forall i \in I, t \in T \quad (6)$$

5. Site Capacity Constraints:

$$\sum_{i \in I} n_{i,t}^{\text{reg}} \leq M \cdot B_{\text{reg}}, \quad \forall t \in T \quad (7)$$

$$\sum_{i \in I} n_{i,t}^{\text{ot}} \leq M \cdot B_{\text{ot}}, \quad \forall t \in T \quad (8)$$

6. Batch Bounds per Coating ID:

$$n_{i,t}^{\text{reg}}, n_{i,t}^{\text{ot}} \in \mathbb{Z}_{\geq 0}, \quad \forall i \in I, t \in T \quad (9)$$

7. Non-negativity of Backlog and Processing Volume:

$$s_{i,t,a}, p_{i,t,a} \geq 0, \quad \forall i \in I, t \in T, a \in A \quad (10)$$

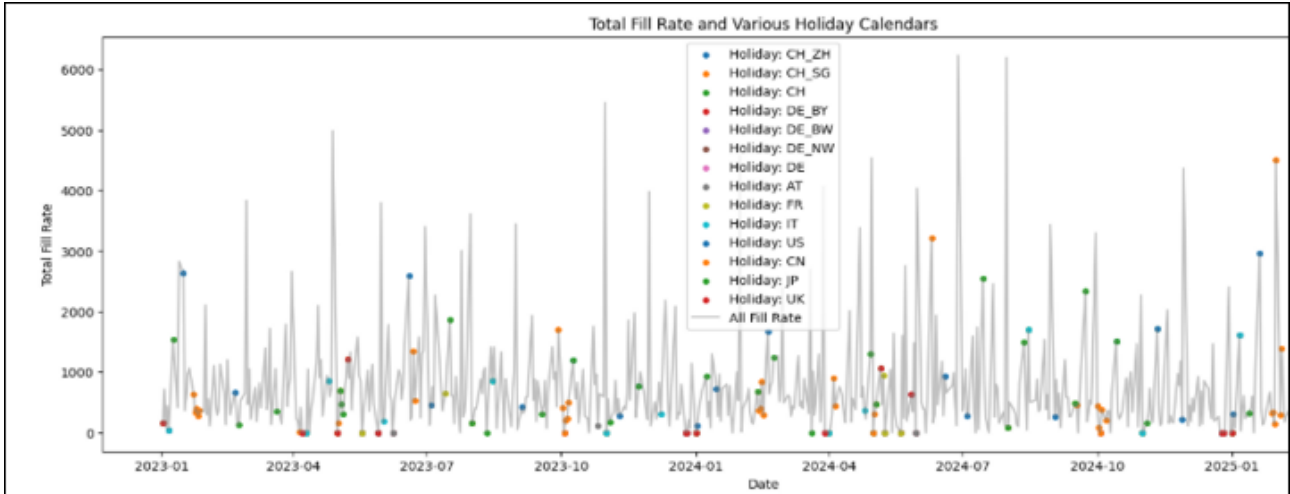


Figure 7 Enter Caption

.1 A.3 Gurobi Data Input Section

```
\begin{lstlisting}[language=Python]
d: Dict[Tuple[str, int], float] = {
    # Actual arrivals for t=0 (today)
    ("C1", 1): 2.2, # Example: Predicted arrivals for C1 on the next
    day
    ("C12", 1): 6.4, # Example: Predicted arrivals for C12 on the next
    day
    ("C15", 1): 2.5, # Example: Predicted arrivals for C15 on the next
    day
}
```

.2 A.4 Holiday Impact on Order Occurrence

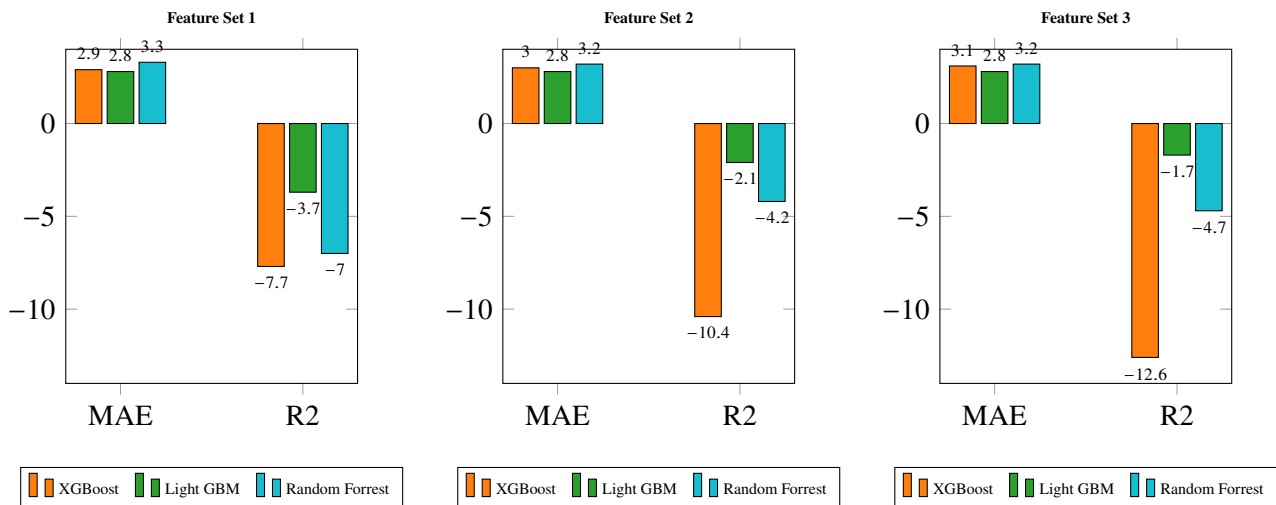


Figure 8 Individual Model per Coating ID

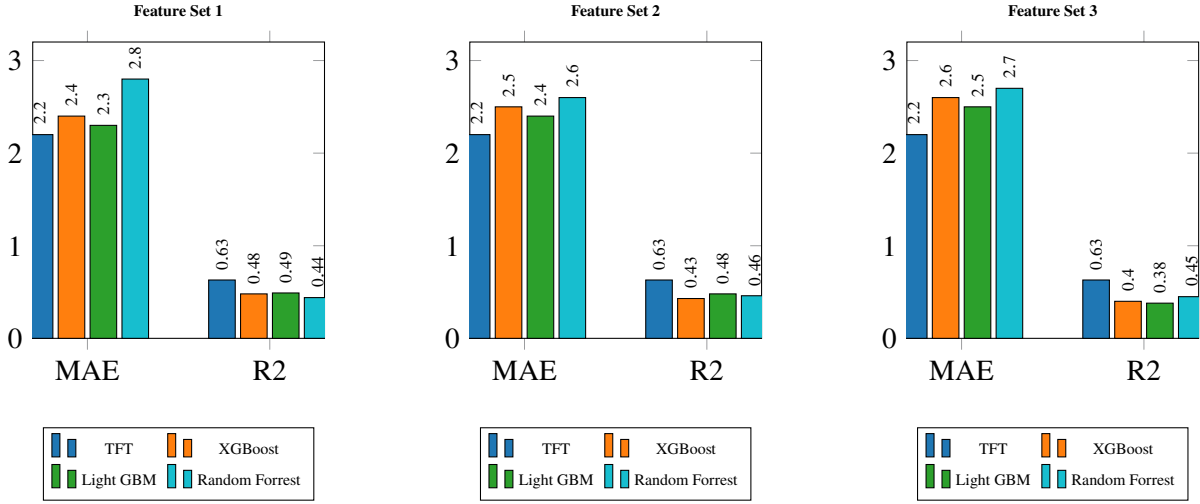


Figure 9 Comparative Performance of all models on the validation set

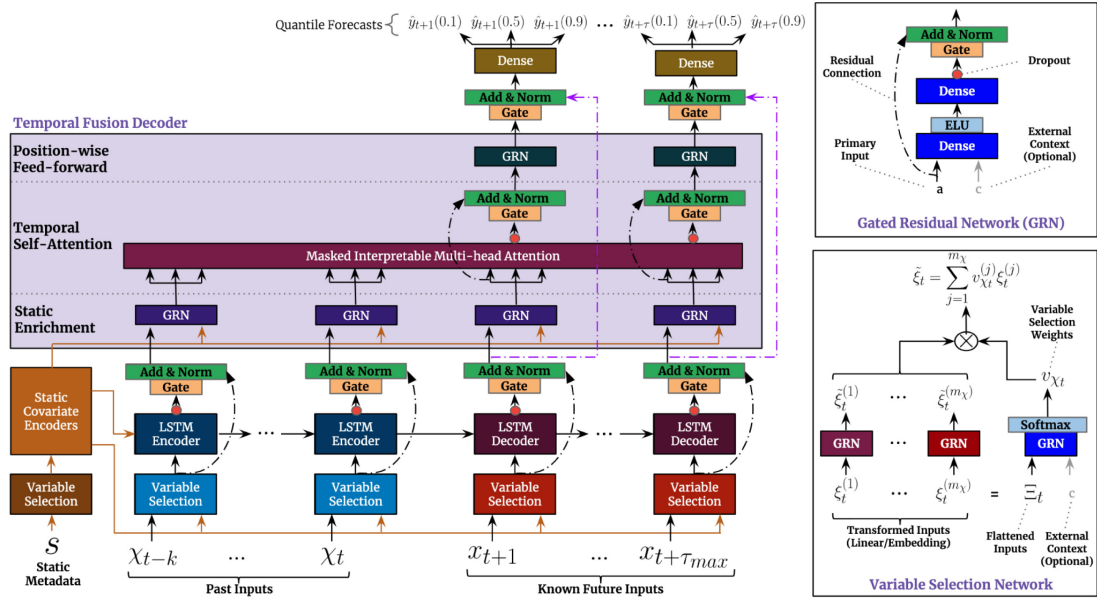


Figure 10 The architecture of the Temporal Fusion Transformer according to [19]

References

- [1] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta, and Masanori Koyama. Optuna: A next-generation hyperparameter optimization framework.
- [2] Elizabeth Barriball, Paul Goodwin, and Richard Spencer. Demand forecasting in the supply chain: A review of models and research directions. *Journal of Business Forecasting*, 36(4):22–28, 2017.
- [3] Yoshua Bengio, Aaron Courville, and Pascal Vincent. Representation learning: A review and new perspectives, 2014.
- [4] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001.
- [5] Tianqi Chen and Carlos Guestrin. XGBoost: A Scalable Tree Boosting System. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*, pages 785–794. ACM, 2016.

- [6] Maximilian Christ, Nils Braun, Julius Neuffer, and Andreas W. Kempa-Liehr. Time series feature extraction on basis of scalable hypothesis tests (tsfresh – a python package). *Neurocomputing*, 307:72–77, 2018.
- [7] Volker Doberanzke. Der einsatz qualitativer methoden der nachfrageprognose bei mittelständischen unternehmen. Working paper 2020-02 (operations management), ZBW–Leibniz Information Centre for Economics, 2020.
- [8] Julia Eichholz, Thorsten Knauer, and Sandra Winkelmann. Digital maturity of forecasting and its impact in times of crisis. *Schmalenbach Journal of Business Research*, 75:443–481, 2023.
- [9] Christodoulos A. Floudas and Xiaoxia Lin. Mixed integer linear programming in process scheduling: Modeling, algorithms, and applications. *Annals of Operations Research*, 139(1):131–162, 2005.
- [10] GeeksforGeeks Contributors. Lightgbm histogram-based learning. <https://www.geeksforgeeks.org/machine-learning/lightgbm-histogram-based-learning/>, 2025. Last updated: July 23, 2025.
- [11] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [12] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on tabular data?, 2022.
- [13] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction, Second Edition*. Springer Series in Statistics. Springer, New York, 2 edition, 2009.
- [14] S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997.
- [15] Rob J Hyndman and George Athanasopoulos. *Forecasting: Principles and Practice*. OTexts, 2nd edition, 2018.
- [16] Rob J Hyndman and George Athanasopoulos. *Forecasting: principles and practice*. OTexts, Melbourne, 2 edition, 2018.
- [17] Azal Ahmad Khan, Omkar Chaudhari, and Rohitash Chandra. A review of ensemble learning and data augmentation models for class imbalanced problems: Combination, implementation and evaluation. *Expert Systems with Applications*, 244:122778, 2024. Open Access.
- [18] L. Li, K. Jamieson, Giulia DeSalvo, A. Rostamizadeh, and A. Talwalkar. Hyperband: A novel bandit-based approach to hyperparameter optimization. *Journal of Machine Learning Research*, 18:1–52, 2018.
- [19] Bryan Lim, Sercan Ö. Arık, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. *International Journal of Forecasting*, 37(4):1748–1764, 2021.
- [20] Xiaojun Mao, Liuhua Peng, and Zhonglei Wang. Nonparametric feature selection by random forests and deep neural networks. *Computational Statistics & Data Analysis*, 170:107436, 2022.
- [21] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15(56):1929–1958, 2014.
- [22] Yu Shi, Guolin Ke, Damien Soukhavong, James Lamb, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, Tie-Yan Liu, Nikita Titov, and David Cortes. *lightgbm: Light Gradient Boosting Machine*, 2025. R package version 4.6.0.99.
- [23] Albert Um. L1, l2 regularization in xgboost regression. <https://albertum.medium.com/l1-l2-regularization-in-xgboost-regression-7b2db08a59e0>, 2021. Medium blog post, published March 21, 2021.
- [24] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need.